

1 chapter3 分治

把规模大的问题分解成小规模问题，要求子问题互相独立且与原问题形式相同，递归解决这些子问题，再把他们的解合并到原问题中，这样的策略称为分治法。

归纳法是尾递归思想，通过扩展子问题解的方式得到原问题的解；而分治法是划分的思想，再合并得到原问题的解。

1.1 分治思想

分治范式通常由以下的步骤组成：

- **划分步骤。**在算法的这个步骤中，输入被分成 $p \geq 1$ 个部分，每个子实例的规模严格小于 n ， n 是原始实例的规模。尽管比 2 大的其他小的常数很普遍，但 p 的最常用的值是 2。 p 也可以具有与 $\log n$ ， n^ε 一样高的数量级 ($0 < \varepsilon < 1$)。
- **治理步骤。**如果问题的规模大于某个预定的阈值 n_0 ，这个步骤由执行 p 个递归调用组成，阈值是由算法的数学分析导出的。
- **组合步骤。**这个步骤是组合 p 个递归调用的解来得到期望的输出值。

分治法所能解决的问题一般具有以下几个特征：

- 该问题的规模缩小到一定的程度就可以容易地解决；
- 该问题可以分解为若干个规模较小的相同问题，即具有最优子结构性质（问题的最优解所包含的子问题的解也是最优的）；
- 利用该问题分解出的子问题的解可以合并为该问题的解；
- 该问题所分解出的各个子问题是相互独立的，即子问题之间不包含公共的子问题。

1.2 分治算法举例

1.2.1 归并排序

如果 n 是 2 的幂时，两个算法的执行比较次数相同。合并两个排好序的子数组，所需要的元素比较次数在 $\frac{n}{2}$ 和 $n - 1$ 之间，所以最小比较次数是：

$$c(n) = 2c\left(\frac{n}{2}\right) + \frac{n}{2}, n \geq 2$$

最多比较次数：

$$c(n) = 2c\left(\frac{n}{2}\right) + n - 1, n \geq 2$$

由主定理知， $c(n) = \Theta(n \log n)$ 如果 n 是任意的正整数，则算法执行元素比较次数是：

$$c(n) = c(\lfloor n/2 \rfloor) + c(\lceil n/2 \rceil) + bn, n \geq 2$$

时间复杂度是 $\Theta(n \log n)$ ，空间复杂度是 $\Theta(n)$ （开出一个临时数组就够用）。

Input: $A[1...n]$

Output: 非降序的 $A[1...n]$

```
1  $t \leftarrow 1$ 
2 while  $t < n$  do
3    $s \leftarrow t; t \leftarrow 2s; i \leftarrow 0$ 
4   while  $i + t \leq n$  do
5      $\text{MERGE}(A, i + 1, i + s, i + t)$ 
6      $i \leftarrow i + t$ 
7   end
8   if  $i + s < n$  then
9      $\text{MERGE}(A, i + 1, i + s, n)$ 
10  end
11 end
```

Algorithm 1: BOTTOMUP-MERGESORT

Input: $A[1...n]$

Output: 非降序的 $A[1...n]$

```
1 if  $low < high$  then
2    $mid \leftarrow \lfloor (low + high) / 2 \rfloor$ 
3    $\text{mergesort}(A, low, mid)$ 
4    $\text{mergesort}(A, mid + 1, high)$ 
5    $\text{MERGE}(A, low, mid, high)$ 
6 end
```

Algorithm 2: MERGESORT

Input : An array A of n elements, a number k such that $1 \leq k \leq n$

Output: The k th smallest element in A

```
1 select(A, 1, n, k)
2 p ← high- low + 1
3 if p < 44 then
4   | sort A and return A [k]
5 end
6 let q = ⌊p/5⌋. Divide A into q groups of 5 elements each. If 5 doesn't divide p, then discard the
   remaining elements.
7 // Although some elements are discarded here, the subsequent grouping still
   takes them into account, so it will not affect the answer.
8 Sort each of the q groups individually and extract its median element. Let the set of medians be
   M
9 mm ← select (M, 1, q, ⌊q/2⌋) // median of medians
10 Partition A into three arrays:
11 A1 ← {A | A < mm}, A2 ← {A | A = mm}, A3 ← {A | A > mm}
12 if |A1| ≥ k then
13   | return select (A1, 1, |A1|, k)
14 else
15   | if k ≤ |A1| + |A2| then
16     | return mm
17   | else
18     | return select (A3, 1, |A3|, k - |A1| - |A2|)
19   | end
20 end
```

Algorithm 3: SELECT algorithm

1.2.2 选择问题：寻找第 k 小元素

设 A'_1 表示小于等于 mm 的元素集， A'_3 表示大于等于 mm 的元素集。所以：

$$|A'_1| \geq 3 \lceil \frac{\lfloor n/5 \rfloor}{2} \rceil \geq \frac{3}{2} \lfloor \frac{n}{5} \rfloor$$
$$|A'_3| \leq n - \frac{3}{2} \lfloor \frac{n}{5} \rfloor \leq n - \frac{3}{2} (\frac{n-4}{5}) = n - 0.3n + 1.2 = 0.7n + 1.2$$

令 $0.7n + 1.2 \leq \lfloor 0.75n \rfloor$ ，解得 $n \geq 44$ (这里的 $\frac{3}{4}$ 选的很好，不到 $\frac{4}{5}$ ，又超过 $\frac{2}{3}$)
所以有：

$$T(n) \leq \begin{cases} c & \text{if } n < 44 \\ T(\lfloor \frac{n}{5} \rfloor) + T(\lfloor \frac{3n}{4} \rfloor) + cn & \text{if } n \geq 44 \end{cases}$$

因而 $T(n) \leq \frac{cn}{1 - \frac{1}{5} - \frac{3}{4}} = 20cn$ 。

对于代码实现，我们可以对快排进行改造：

```

1 void qsort(int l, int r){
2     // if(l >= r) return;
3     // 这个必须要注释掉，否则就可能不输出了
4     int i = l, j = r, x = a[(l + r) >> 1];
5     while(i <= j){
6         while(a[i] < x) i++;
7         while(a[j] > x) j--;
8         if(i <= j){
9             swap(a[i], a[j]);
10            i++; j--;
11        }
12    }
13    if(k <= j) qsort(l, j);
14    else if(k >= i) qsort(i, r);
15    else{
16        printf("%d\n", a[j + 1]);
17        exit(0);
18    }
19 }

```

快排是需要 $O(n)$ 的空间来存储递归中的左右边界的。运行时间最坏情况下为 $\Theta(n^2)$ ，整体复杂度为 $\Theta(n \log n)$ 。

1.2.3 最值选择算法的改进

原始 MINMAX 算法 原始算法要求输入规模 n 必须为 2 的幂次 ($n = 2^k$)，其伪代码如下：

```

1 Function MinMax( $A, low, high$ )
2      $mid \leftarrow \lfloor (low + high)/2 \rfloor$ ;
3     if  $high - low = 2$  then
4         return ( $A[low], A[high]$ ) if  $A[low] < A[high]$  else ( $A[high], A[low]$ );
5     end
6      $(x_1, y_1) \leftarrow \text{MinMax}(A, low, mid)$ ;
7      $(x_2, y_2) \leftarrow \text{MinMax}(A, mid, high)$ ;
8     return ( $\min(x_1, x_2), \max(y_1, y_2)$ );

```

Algorithm 4: Original MinMax Algorithm (Requires $n = 2^k$)

时间复杂度分析：

- 比较次数递推式：

$$C(n) = \begin{cases} 1 & n = 2 \\ 2C(n/2) + 2 & n > 2 \end{cases}$$

- 精确解： $C(n) = \frac{3n}{2} - 2$

改进后的 MINMAX 算法 改进算法通过以下修改支持任意正整数 n :

```

1 Function MinMax( $A, low, high$ )
2    $mid \leftarrow \lfloor (low + high)/2 \rfloor$ ;
3   if  $high - low = 1$  then
4     return ( $A[low], A[low]$ );
5   else if  $high - low = 2$  then
6     return ( $A[low], A[high]$ ) if  $A[low] < A[high]$  else ( $A[high], A[low]$ );
7    $(x_1, y_1) \leftarrow \text{MinMax}(A, low, mid)$ ;
8    $(x_2, y_2) \leftarrow \text{MinMax}(A, mid, high)$ ;
9   return ( $\min(x_1, x_2), \max(y_1, y_2)$ );

```

Algorithm 5: Modified MinMax Algorithm (Works for any $n \in \mathbb{N}^+$)

改进要点分析

- 新增递归基：处理 $n = 1$ 的情况（第 3-4 行）
- 非对称划分：采用 $\lfloor n/2 \rfloor$ 和 $\lceil n/2 \rceil$ 划分
- 通用性：支持任意正整数输入规模

比较次数分析 改进后的比较次数满足递推关系：

$$T(n) = \begin{cases} 0 & n = 1 \\ 1 & n = 2 \\ T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 2 & n > 2 \end{cases}$$

理论结论

- 改进算法比较次数趋近于 $\lceil \frac{3n}{2} \rceil - 2$ 但非严格相等
- 当 $n = 2^k$ 时与原始算法完全一致
- 最坏情况下比较次数上界为 $3\lceil n/2 \rceil - 2$

1.2.4 有序矩阵找第 k 小的数

给定一个 $n \times n$ 的矩阵，其中每一行和每一列的元素均按升序排列。需要找到矩阵中第 k 小的元素。

1. 二分查找区间：初始区间为矩阵的最小值和最大值：

$$l = \text{matrix}[0][0], \quad r = \text{matrix}[n-1][n-1]$$

2. 统计小于等于 mid 的元素个数：

- 对于中点 mid ，统计矩阵中 $\leq mid$ 的元素个数 $count$
- 利用每行有序的性质，在每行中用二分查找（upper_bound）统计

3. 调整区间:

$$\text{count} < k \Rightarrow l = \text{mid} + 1$$
$$\text{count} \geq k \Rightarrow r = \text{mid} - 1$$

4. 终止条件: 当 $l > r$ 时, l 即为第 k 小的元素

- 时间复杂度: $O(n \log n \log C)$
 - 二分查找次数: $O(\log C)$ ($C = \max - \min$)
 - 每次统计 count: $O(n \log n)$
- 空间复杂度: $O(1)$

Input: A sorted $n \times n$ matrix where each row and column is sorted in ascending order, and an integer k

Output: The k -th smallest element in the matrix

```
1  $n \leftarrow \text{len}(\text{matrix});$ 
2  $l \leftarrow \text{matrix}[0][0];$ 
3  $r \leftarrow \text{matrix}[n-1][n-1];$ 
4 while  $l \leq r$  do
5    $\text{mid} \leftarrow l + \lfloor \frac{r-l}{2} \rfloor;$ 
6    $\text{count} \leftarrow 0;$ 
7   for  $i \leftarrow 0$  to  $n-1$  do
8      $\text{count} \leftarrow \text{count} + \text{bisect\_right}(\text{matrix}[i], \text{mid});$ 
9   end
10  if  $\text{count} < k$  then
11     $l \leftarrow \text{mid} + 1;$  // 第  $k$  小元素在右半区间
12  else
13     $r \leftarrow \text{mid} - 1;$  // 第  $k$  小元素在左半区间
14  end
15 end
16 return  $l;$ 
```

Algorithm 6: Find K-th Smallest Element in Sorted Matrix

1.2.5 多数元素问题

根据多数元素的定义可知多数元素必定在数组中出现次数最多, 而若将原数组分成 2 部分, 则在其中一部分出现次数最多的必定是多数元素。

该算法的时间复杂度是 $O(n \log n)$, 空间复杂度是 $O(\log n)$ 。

1.2.6 芯片测试问题

问题描述：有 n 片芯片，已知其中好芯片至少比坏芯片多 1 片，需要通过测试从中找出 1 片好芯片。如何设计一个算法，使用最少测试次数，从中挑出 1 片好芯片？

测试方法：将芯片放到测试台上，2 片芯片互相测试，并报告结果“好”或“坏”。好芯片报告结果一定是正确的，坏芯片的报告不可靠（可能对也可能不对）。

如果 n 是偶数，那么将 n 片芯片两两一组做淘汰测试，剩下芯片构成子问题，进入下一轮分组淘汰。

淘汰规则：如果报告结果为“好，好”，则任留 1 片，否则全部抛弃，对于 $n = 3$ 单独测试，对于 $n = 2, n = 1$ 的情况，则任取其一就是好芯片。

要证明分治算法的正确性，只需要证明子问题与愿问题性质相同且算法是有退出的出口的。

如果 n 是奇数，则任选 1 个芯片，用其他 $n - 1$ 个芯片对他进行测试，若是好芯片，则算法退出，否则丢弃他，转化为是偶数个的情况。

设 $W(n)$ 是最坏情况下的测试次数，那么有：

$$W(n) = W\left(\frac{n}{2}\right) + O(n)$$

由主定理知， $W(n) = O(n)$

1.2.7 循环赛日程表

设有 $n = 2^k$ 个运动员要进行网球循环赛。现要设计一个满足以下要求的比赛日程表：

1. 每个选手必须与其他 $n-1$ 个选手各赛一次；
2. 每个选手一天只能赛一次；
3. 循环赛一共进行 $n-1$ 天。

```
1 void sol(int x, int y, int size, int st){
2     if(size >= 2){
3         sol(x, y, size - 1, st);
4         sol(x + (1 << (size - 1)), y, size - 1, st + (1 << (size - 1)));
5         sol(x, y + (1 << (size - 1)), size - 1, st + (1 << (size - 1)));
6         sol(x + (1 << (size - 1)), y + (1 << (size - 1)), size - 1, st);
7     }
8     else{
9         plan[x][y] = plan[x + 1][y + 1] = st;
10        plan[x + 1][y] = plan[x][y + 1] = st + 1;
11    }
12 }
```

1.2.8 向量卷积

给定向量 $a = (a_0, a_1, \dots, a_n), b = (b_0, b_1, \dots, b_n)$, 则有二者的卷积为 $c = (c_0, c_1, \dots, c_n)$, 其中

$$c_i = \sum_{j=0}^i a_j b_{i-j}$$

而卷积与多项式乘法等价, 即令 $A(x) = a_0 + a_1x + \dots + a_{m-1}x^{m-1}, B(x) = b_0 + b_1x + \dots + b_{n-1}x^{n-1}$, 则 $C(x) = A(x)B(x) = a_0b_0 + (a_0b_1 + a_1b_0)x + \dots + a_{m-1}b_{n-1}x^{m+n-2}$

如果采用暴力做法, 先逐项相乘, 再同类合并, 那么复杂度将是 $O(n^2)$.

我们设计特定值 $x_1, x_2, \dots, x_{2n}$ 求出 $A(x_j)$ 和 $B(x_j), j = 1, 2, \dots, 2n$, 进一步地计算出 $C(x_j)$, 最后利用多项式插值方法, 根据 $C(x)$ 在 $x = x_1, x_2, \dots, x_{2n}$ 的插值点, 求出 $C(x)$.

而基于此, 我们最关键的计算提速步骤就在于利用分治算法的思想对 $A(x_i)$ 的求值步骤进行加速:

$$\begin{aligned} A_{\text{even}}(x) &= a_0 + a_2x + a_4x^2 + \dots + a_{n-2}x^{\frac{n-2}{2}} \\ A_{\text{odd}}(x) &= a_1 + a_3x + a_5x^2 + \dots + a_{n-1}x^{\frac{n-2}{2}} \\ A(x) &= A_{\text{even}}(x^2) + xA_{\text{odd}}(x^2) \end{aligned}$$

这里相当于把本来 x 是 1 的 $2n$ 次方根, 变成了 x^2 是 1 的 n 次方根, 实现了问题的降级, 从根本上使得复杂度从 $O(n^2)$ 变成了 $O(n \log n)$.

而在这个过程中, 我们利用复数单位根的对称性可以进一步地加速, 即:

$$\begin{aligned} A(w_j) &= A_{\text{even}}(w_j^2) + w_j A_{\text{odd}}(w_j^2) \\ A(w_{j+n}) &= A_{\text{even}}(w_j^2) - w_j A_{\text{odd}}(w_j^2) \end{aligned}$$

而如何通过 $C(w_i)$ 求出 c_i , 方法如下, 但是不要深究。

构造多项式:

$$D(x) = C(w_0) + C(w_1)x + \dots + C(w_{2n-1})x^{2n-1}$$

可以证明 (别管怎么证的):

$$D(w_i) = 2nc_{2n-j}$$

至此, 时间复杂度被优化到了 $O(n \log n)$

1.2.9 棋盘覆盖

其实就是这个题, 分治思想: <https://www.luogu.com.cn/problem/P1228>

由此可想到将一个边长为 2^k 的正方形分成四个边长为 2^{k-1} 的正方形, 特殊方格必在其中的一个正方形中。然后再中心摆上一个地毯, 地毯缺少的一块朝向特殊方格所在正方形, 这样相当于每个小正方形中有一个特殊方格。一直分治下去, 直到边长为 2 结束。

1.2.10 最大子序列和问题

题目: <https://www.luogu.com.cn/problem/P1115>

1. 第一个数为一个有效序列
2. 如果一个数加上上一个有效序列得到的结果比这个数大, 那么该数也属于这个有效序列。

3. 如果一个数加上上一个有效序列得到的结果比这个数小，那么这个数单独成为一个新的有效序列
4. 在执行上述处理的过程中实时更新当前有效序列的所有元素之和并取最大值。

这样代码就是：

```
1  for(int i=0;i<n;i++){
2      cin>>a;
3      f=max(f+a,a);
4      ans=max(f,ans);
5  }
```

而如果用分治算法解决这道题，只要求出左半部分和右半部分的最大子序列和，最后计算一定要包含中间的最大子序列和，三者取最大值就是答案。

```
1  int g(int l,int m,int r){//l,右边界、r,左边界、m,中间数
2      int max1=a[m],max2=a[m+1];//最大值
3      int sum1=0,sum2=0;//和，循环用
4      //从m向左延伸
5      for(int i=m;i>=l;i--){
6          sum1+=a[i];
7          max1=max(max1,sum1);
8      }
9      //从m+1向右延伸
10     for(int i=m+1;i<=r;i++){
11         sum2+=a[i];
12         max2=max(max2,sum2);
13     }
14     return max1+max2;
15 }
16
17 int he(int l,int r){
18     if(l==r){
19         return a[l];
20     }
21     int mid=(l+r)/2;
22     return max(he(l,mid),max(he(mid+1,r),g(l,mid,r)));
23 }
```

1.3 分治算法的改进

主要就是两个渠道：减少子问题数量、减少递归内部计算量。

1.3.1 大整数乘法

设 n 是 2 的幂, $u = w \cdot 2^{\frac{n}{2}} + x, v = y \cdot 2^{\frac{n}{2}} + z$, 那么 $uv = wy2^n + (xy + wz)2^{\frac{n}{2}} + xz$, 但是这样时间复杂度是 $\Theta(n^2)$ 的。

我们可以考虑 $wz + xy = (w + x)(y + z) - wy - xz$, 这样就只需要进行 3 个 $\frac{n}{2}$ 的子问题了, 复杂度为 $\Theta(n^{\log 3}) = O(n^{1.59})$

1.3.2 矩阵乘法

暴力做法是 $\Theta(n^3)$ 的, 而如果只是粗暴的分区处理, 仍然是 $\Theta(n^3)$ 的, 这里我们介绍 STRASSEN 算法, 增加加减法的次数来减少乘法次数。

1. 与朴素分治算法相同, 分解出左上, 左下, 右上, 右下四个子矩阵。
2. 创建 10 个 $\frac{n}{2} \times \frac{n}{2}$ 的矩阵 S_i , 每个 S_i 保存两个子矩阵的和或差。
3. 用子矩阵和 S_i 相乘, 递归地计算 7 个 $\frac{n}{2} \times \frac{n}{2}$ 的矩阵 P_i 。
4. 通过 P_i 的不同组合进行加减, 得到 C 的子矩阵。

$$\begin{aligned}
 S_1 &= B_{1,2} - B_{2,2} & S_2 &= A_{1,1} + A_{1,2} \\
 S_3 &= A_{2,1} + A_{2,2} & S_4 &= B_{2,1} - B_{1,1} \\
 S_5 &= A_{1,1} + A_{2,2} & S_6 &= B_{1,1} + B_{2,2} \\
 S_7 &= A_{1,2} - A_{2,2} & S_8 &= B_{2,1} + B_{2,2} \\
 S_9 &= A_{1,1} - A_{2,1} & S_{10} &= B_{1,1} + B_{1,2} \\
 P_1 &= A_{1,1} \cdot S_1 & P_2 &= S_2 \cdot B_{2,2} \\
 P_3 &= S_3 \cdot B_{1,1} & P_4 &= A_{2,2} \cdot S_4 \\
 P_5 &= S_5 \cdot S_6 & P_6 &= S_7 \cdot S_8 \\
 P_7 &= S_9 \cdot S_{10} \\
 C_{1,1} &= P_5 + P_4 - P_2 + P_6 & C_{1,2} &= P_1 + P_2 \\
 C_{2,1} &= P_3 + P_4 & C_{2,2} &= P_5 + P_1 - P_3 - P_7
 \end{aligned}$$

对于矩阵规模不是 2 的幂的情况, 只需要补 0 即可。

类似地, 试只用三次乘法完成复数相乘 $(a + bi)(c + di) = (ac - bd) + (ad + bc)i$ 。

设

$$\begin{cases} \alpha = ac \\ \beta = bd \\ \gamma = (a + b)(c + d) \end{cases}$$

则:

$$(a + bi)(c + di) = (\alpha - \beta) + (\gamma - \alpha - \beta)i$$

1.3.3 最近点对问题

方法是前面提到过的, 按找 x 坐标排序后割开成两部分, 最终答案就是 $\delta_l, \delta_r, \delta'$ 之间的最小值。但是关键的问题在于 δ' (左右之间的最小值) 该怎么求。(直接用前面求出来的 $\delta = \min(\delta_l, \delta_r)$, 这样得到一个高为 δ , 长为 2δ 的矩形, 显然对于一个正方形来说, 两两距离超过边长的点最多有 4 个, 所以这个最多有 8 个, 这样只需要枚举其附近的 7 个点即可。

1.4 本章作业

1.4.1 快排空间复杂度

证明算法 QUICKSORT 的空间复杂度在 $\Theta(\log n)$ 与 $\Theta(n)$ 之间变动。并计算该算法的平均空间复杂度是多少？

快速排序的空间复杂度主要由递归调用栈的深度（栈中存储的是枢轴量）决定，而递归深度取决于划分的平衡性。

快速排序的平均空间复杂度由递归树的平均深度决定。通过分析划分的期望行为，可以证明平均深度为 $O(\log n)$ 。

划分点是随机选择的，划分后左右子数组大小的比例为任意 $(k, n - k - 1)$ 的概率均等。设 $D(n)$ 为递归树深度的期望值，满足以下递推关系：

$$D(n) = 1 + \frac{1}{n} \sum_{k=0}^{n-1} \max(D(k), D(n - k - 1))$$

由于 $D(k)$ 和 $D(n - k - 1)$ 对称，可简化为：

$$D(n) = 1 + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} D(k)$$

假设 $D(k) \leq c \log k$ 对所有 $k < n$ 成立，则：

$$D(n) \leq 1 + \frac{2c}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} \log k$$

通过积分近似求和（ $\sum \log k \approx n \log n - n$ ），可得：

$$D(n) \leq 1 + \frac{2c}{n} \left(n \log n - \frac{n}{2} \log \frac{n}{2} \right) \approx c \log n$$

因此， $D(n) = O(\log n)$ 。

1.4.2 多选问题

设 S 是包含 n 个不等数的集合，给定整数 r ($1 \leq r \leq n$)， $K = \{k_1, k_2, \dots, k_r\}$ 且 $1 \leq k_1 < k_2 < \dots < k_r \leq n$ 。要求在 S 中找出第 k_1 小，第 k_2 小， $\dots$ ，第 k_r 小的 r 个元素。例如， $r = 3$ ， $K = \{2, 4, 7\}$ ，则输出 S 中的第 2 小，第 4 小，第 7 小的元素。当 $r = 1$ 时，该问题就是一般选择问题。当 $r = n$ 时，该问题就变成了排序问题。

1. 设计一个最坏情况下时间复杂度为 $O(nr)$ 的算法。
2. 若 $r > 1$ ，设计一个最坏情况下时间复杂度为 $O(n \log r)$ 的算法。

时间复杂度 $O(nr)$ 的算法

时间复杂度 $O(n \log r)$ 的算法

1. 基本情况处理：

- 若 $r = 1$ ，直接调用 SELECT 算法（时间复杂度 $O(n)$ ）返回第 k_1 小的元素

Input: 集合 S 包含 n 个不等数, 整数 r , 集合 $K = \{k_1, k_2, \dots, k_r\}$ 且

$$1 \leq k_1 < k_2 < \dots < k_r \leq n$$

Output: 集合 S 中第 k_1 小, 第 k_2 小, $\dots$, 第 k_r 小的 r 个元素

```
1 对  $K$  进行排序, 确保  $k_1 < k_2 < \dots < k_r$ ;  
2 初始化结果数组  $R[1..r]$ ;  
3 for  $i \leftarrow 1$  to  $r$  do  
4   | 使用 SELECT 算法在  $S$  中找出第  $k_i$  小的元素  $x$ ;  
5   |  $R[i] \leftarrow x$ ;  
6 end  
7 return  $R$ ;
```

Algorithm 7: MultipleSelectBasic

2. 预处理:

- 对 K 排序, 确保 $k_1 < k_2 < \dots < k_r$

3. 选取枢纽元:

- 计算 K 的中位数位置: $k_{mid} = k_{\lfloor r/2 \rfloor}$
- 使用 SELECT 算法在 S 中找到第 k_{mid} 小的元素 $pivot$

4. 划分数组:

- $L = \{x \in S \mid x < pivot\}$
- $E = \{x \in S \mid x = pivot\}$
- $G = \{x \in S \mid x > pivot\}$

5. 递归处理子问题:

- 左子问题 $K_L = \{k_i \in K \mid k_i < k_{mid}\}$: 在 L 中递归求解
- 右子问题 $K_G = \{k_i \in K \mid k_i > k_{mid}\}$: 在 G 中递归求解, 调整查询排名 $k'_i = k_i - |L| - |E|$

6. 合并结果:

- 返回 $R_L \cup \{pivot\} \cup R_G$

时间复杂度分析

- 递归深度: $O(\log r)$ (每次 K 规模减半)
- 每层工作量: $O(n)$ (SELECT 和划分)
- 总时间复杂度: $O(n \log r)$

Input: 集合 S 包含 n 个不等数, 整数 $r > 1$, 集合 $K = \{k_1, k_2, \dots, k_r\}$ 且

$$1 \leq k_1 < k_2 < \dots < k_r \leq n$$

Output: 集合 S 中第 k_1 小, 第 k_2 小, $\dots$, 第 k_r 小的 r 个元素

```
1 if  $r = 1$  then
2   |   return SELECT( $S, k_1$ );
3 end
4 对  $K$  进行排序, 确保  $k_1 < k_2 < \dots < k_r$ ;
5  $k_{mid} \leftarrow k_{\lfloor r/2 \rfloor}$ ;
6  $pivot \leftarrow$  SELECT( $S, k_{mid}$ );
7 将  $S$  划分为:  $L \leftarrow \{x \in S \mid x < pivot\}$ ;
8  $E \leftarrow \{x \in S \mid x = pivot\}$ ;
9  $G \leftarrow \{x \in S \mid x > pivot\}$ ;
10 初始化子问题的  $K$  集合:  $K_L \leftarrow \{k_i \in K \mid k_i < k_{mid}\}$ ;
11  $K_G \leftarrow \{k_i \in K \mid k_i > k_{mid}\}$ ;
12 递归解决子问题:  $R_L \leftarrow$  MultipleSelectOptimized( $L, K_L, |L|, |K_L|$ );
13  $R_G \leftarrow$  MultipleSelectOptimized( $G, K_G - |L| - |E|, |G|, |K_G|$ );
14 return  $R_L \cup \{pivot\} \cup R_G$ ;
```

Algorithm 8: MultipleSelectOptimized